Verdict Attestations: A Cross-Registry, Offline-Verifiable Ledger for AI Agent Component Supply Chains

Canari Contributors
github.com/lanbass869-cell/canari

November 2026

Abstract

AI agent supply chains face a gap between two existing trust layers: registry-native identity attestations (Sigstore provenance for npm, PEP 740 for PyPI) prove which code was built, while security scanners enumerate what patterns the code contains. Neither layer produces a portable, offline-verifiable, time-bound verdict about whether a component is safe to install, and no existing system unifies verdicts across npm, PyPI, the Model Context Protocol (MCP) registry, and community plugin registries.

We present Canari, a cross-registry verification ledger that produces signed DSSE/Ed25519 attestation objects binding verdicts to artifact hashes, with bi-temporal timestamps, supersession chains, and same-author revocation propagation. Canari does not compete with identity attestations (it consumes them as input) or with deep scanners (it orchestrates them as adapters); it fills the verdict layer between them. We demonstrate the system with 294 attestations spanning four registries, a live-verified npm provenance ingestion path (sigstore-python 4.5.1 against express@5.3.0), and a PyPI native attestation pipeline that is ready for adoption (measured coverage: 0/40 recent uploads). We publish all code, data, and schemas as open artifacts.

1 Introduction

LLM-based agents increasingly rely on third-party components: MCP servers, agent skills, and plugins from community registries. Two threats dominate this supply chain. First, the tool update gap: trust established for a tool does not remain valid after implementation updates. Kan et al. [1] demonstrated a 97.6% attack success rate by embedding malicious functionality into updates of initially benign tools, bypassing prompt-injection detectors and static skill scanning entirely. Second, silent drift: half of multi-version MCP servers (51.1%) changed what they advertise between versions, 40.6% did so silently, and 4.2% redirected their remote endpoint to a different host while keeping the same registry identity [2].

The ecosystem has developed two families of defenses, but a gap remains between them:

1. Identity attestations (Sigstore provenance on npm, PEP 740 on PyPI) bind a release to its builder identity. These prove which code runs but not what it does—Kan et al. [1] states this explicitly: "hashes and attestations certify which code runs, never what it does." As of November 2026, 60% of top-30 npm packages carry Sigstore provenance, but PyPI coverage is effectively zero (0/30 top packages, 0/40 fresh uploads measured).

2. Security scanners (NVIDIA SkillSpector [8], Cisco skill-scanner [9], Snyk agent-scan [10], repo-forensics [11]) enumerate vulnerability patterns in source code. The deepest open-source scanner we found ships 1,138 regex rules. These tools produce findings but not signed, portable, time-bound verdicts, and they do not track drift across versions.

The gap. No existing system produces signed, offline-verifiable verdict attestations that are (a) portable across registries, (b) linked into supersession chains so history is never lost, (c) propagated through same-author revocation, and (d) consumable by a client-side install gate. Closest systems fall short: mcpcheckup [4] provides signed monitoring for remote MCP servers only; SIGIL [5] gates skills through an on-chain registry (imposing chain infrastructure); agtmls [6] maintains a signed skill registry with revocation but for a single registry; and the manifest signing of Jamshidi et al. [7] operates within MCP pipelines without cross-registry scope.

2 System Design

2.1 Attestation Format

Each attestation is a JSON payload signed with DSSE [13] over an Ed25519 keypair. The schema (v0.1) is a domain-specific in-toto predicate. Core fields:

- subject: {name, version, registry, artifact_sha256} — binds the verdict to exact bytes, not to a mutable name.
- scenario_sha256: SHA-256 of the check-list definition; changing the scanner invalidates prior verdicts.
- checks[]: per-check verdict (pass/fail/skip/unsupported/error) with tool identifier.
- verdict: run-level summary (pass, pass-with-findings, fail, unsupported, error).
- supersedes: digest of the superseded predecessor (supersession chain).
- related_revocations[]: same-author / name-variant / dependency links to revocation events (propagation).
- native_provenance: ingested registry-native attestation result (presence + verification outcome), recorded as an input signal, never as a safety verdict.
- evidence_completeness: explicit disclosure of missing evidence (anti-inflation discipline learned from dsh-testkit: unsupported $\neq$ failed $\neq$ pass).

2.2 Supersession Chains

When a component is re-scanned, the new attestation carries supersedes pointing to the digest of the previous attestation for the same subject. Chains are append-only: old verdicts are never deleted, so an auditor can reconstruct the full history of a component's trust state. This mirrors the supersession semantics of MemTether [14] (a cross-client memory hub) adapted to supply-chain objects.

2.3 Same-Author Revocation Propagation

When a component is revoked (e.g., a registry yank, a scanner fail, or a maintainer removal), Canari queries the state store for all components sharing the same author, name variants, or declared dependencies, and appends related_revocations entries to their next attestations. This is annotation, not judgment: the consumer decides whether a related revocation affects their install decision.

2.4 Native Attestation Ingestion

Canari distinguishes between identity attestations (registry-native) and verdict attestations (Canari's own). Native provenance is consumed as an input signal:

- npm: the Sigstore bundle (v1, certificate-based) is verified offline via sigstore-python 4.5.1's Verifier.verify_dsse(). Verified live against express@5.3.0: DSSE verification passes, identity is extracted from the certificate SAN (https://github.com/expressjs/express/.github/workflows/npm-publish.yml@refs/tags/v5.3.0). v0.1 bundles (publicKey hint, no certificate chain) are recorded as presence-only; no fake verification is attempted.
- PyPI: PEP 740 objects are converted to Sigstore Bundles via pypi-attestations 0.0.31 and verified identically. Measured adoption is currently zero (0/30 top, 0/40 fresh), but the pipeline is ready.

2.5 Scanner Orchestration

Canari does not build its own vulnerability scanner. L1–L2 checks (registry present, version resolved, artifact hash, maintainer set, manifest fields) are meta-only. L3+ behavioral checks are delegated to external scanners (sentinel, repo-forensics) via a scanner-adapter interface that records: scanner name + version, rule count, findings JSON, and a digest of the scanner source tree (scenario binding). This keeps Canari's core small (∼300 lines of Python) while allowing deep scans from the best available tools.

3 Implementation

Canari is implemented in Python 3.13 with the following dependencies: in-toto 3.1.0 (DSSE envelopes), securesystemslib 1.6 (Ed25519), jsonschema (schema validation), sigstore-python 4.5.1 (native npm verification), pypi-attestations 0.0.31 (PyPI native). The codebase is ∼1,200 lines of Python across 8 modules, with 53 tests (52 passed, 1 skipped for no live PyPI attestation available).

Adapters exist for four registries:

- npm (packument + tarball download for hash, search for discovery, native provenance via dist.attestations)
- PyPI (JSON API + Simple API for full enumeration)
- MCP Registry (registry.modelcontextprotocol.io, cursor-paginated)
- dsh.so (market index with 10,195 plugins, per-plugin install-test and security-scan endpoints, CC-BY-4.0 licensed)

4 Evaluation

4.1 Coverage

The MCP Registry contains at least 50,000 entries (server + version records) spanning 19,330 distinct server names (live count, November 2026, full cursor pagination). A multi-registry run produced 294 attestations (npm: 100, PyPI: 94, MCP Registry: 100) with 99.6% schema integrity and zero hidden errors. Failure counts are publicly recorded in the feed (dead-letter audit).

Table 1: Native attestation coverage (measured 2026-11-08).

Registry	Sample	Coverage
npm (top-30)	30	18/30 (60%)
PyPI (top-30)	30	0/30 (0%)
PyPI (fresh uploads)	40	0/40 (0%)

4.2 Native Provenance

Table 1 summarizes measured native attestation coverage.

The npm v1 bundle for express@5.3.0 verifies: DSSE signature valid, identity = npm-publish.yml workflow at tag v5.3.0, tlog entry present. The v0.1 bundle (publicKey hint) is recorded as presence-only. On PyPI, zero coverage means Canari's PEP 740 pipeline has no live test target yet; the blank itself is a finding.

4.3 Drift Cases

We use the public MCP Registry Drift Panel [3] (Zenodo 21798111, CC-BY-4.0) as baseline data. The panel records 18,966 distinct servers over 88.6 days. Key statistics: 75.2% of servers never change; the most active 5% generate 61% of all change events; drift-ranked re-auditing at a 5% budget catches only ∼20% of drifts. Canari's contribution is the verdict layer on top of this measurement data: for each drift event, Canari produces a signed re-attestation whose supersedes pointer makes the drift auditable and reversible.

4.4 Propagation

Five test cases verify the same-author revocation propagation logic: same-author detection, name-variant matching, no self-revocation, empty set, and relation-field correctness. The propagation is annotation-only; the consumer's install gate decides.

4.5 Honest Scoring

Canari publishes a four-dimension score with every run: coverage (attested / enumerated), freshness (within TTL), integrity (signature + schema pass rate), and failure transparency (error counts). All scores include sample size, calculation method, and known distortions — a score without these is a lie.

5 Related Work

Table 2 compares Canari with the closest systems.

6 Limitations

1. Scan depth zero. Canari's built-in checks are meta-only (L1–L2). Deep vulnerability detection requires external scanners (L3+). This is by design: Canari is a ledger, not a scanner. The adapter interface makes this composable.

2. Single signer. All attestations in the current deployment are signed by one key. A production system would need multi-signer support or a transparency log for the signer keys themselves.

3. PyPI blank. Zero native attestation coverage means the PyPI ingestion path has no live verification target. The pipeline is ready but untested against real data.

Table 2: Canari vs. the closest systems (verified against source code, not marketing).

System	Cross-registry	Signed	Offline	Supersession	Propagation	
mcpcheckup [4]	no (MCP only)	yes	yes	no	no	
SIGIL [5]	no (skills)	yes (chain)	no (chain)	no	no	y
agtmls [6]	no (own registry)	yes	yes	no	no (advisory)	
Manifest signing [7]	no (MCP pipeline)	yes	yes	no	no	
Native provenance (npm/PyPI)	per-registry	yes	yes	no	no	
SkillSpector [8]	no	no	–	no	no	
repo-forensics [11]	no	no	–	no	no	
trustcheck [12]	no (PyPI)	yes (PEP 740)	yes	no	no	
Canari	yes	yes	yes	yes	yes	

4. MCP Registry count is a floor. The live API caps pagination at 400 pages (∼40,000 entries including versions); distinct-server count requires resuming pagination with backoff.

5. npm v0.1 bundles unverifiable with off-the-shelf tools. sigstore-python 4.5.1's Bundle model requires a certificate chain; v0.1 publicKey-hint bundles are recorded as presence-only. A custom DSSE verifier could close this gap.

7 Conclusion

Canari fills a specific gap in the AI agent supply chain: the verdict layer between identity attestations (which prove who built the code) and deep scanners (which enumerate what patterns the code contains). Its contribution is the combination of cross-registry scope, offline-verifiable signed attestations, supersession chains, same-author revocation propagation, and a consumer-side install gate — four capabilities that no existing system provides together. All code, schemas, and data are open artifacts.

References

[1] S. Kan, Y. Qi, Y. Han, X. Luo, G. Xiong, X. Gao. Beyond Prompt Injection: The Tool Update Gap in LLM Agent Security. arXiv:2611.01976, 2026.

[2] O. Kraishan. Same Name, Different Server: A Security Census of Silent Drift in the Model Context Protocol Ecosystem. arXiv:2609.14119, 2026.

[3] G. Bharti. Registry Descriptions Go Stale Unevenly: An 89-Day Measurement of MCP Drift. arXiv:2608.00997, 2026. Data: zenodo.org/records/21798111.

[4] MCP Checkup. github.com/mcpcheckup/checks, 2026. Live: mcpcheckup.com.

[5] T. Shen et al. Sealing the Audit-Runtime Gap for LLM Skills (SIGIL). arXiv:2605.05274, 2026.

[6] S. Rousseau. AgtMLS: Signed, Audited Agent Skills Registry. github.com/sebastienrousseau/agtmls, 2026.

[7] S. Jamshidi et al. Verifiable Manifest Signing and Transparency Enforcement for Secure MCP-Based LLM Pipelines. arXiv:2601.23132, 2026.

[8] NVIDIA. SkillSpector: Security Scanner for AI Agent Skills. github.com/NVIDIA/SkillSpector, 2026.

[9] Cisco AI Defense. Skill Scanner / MCP Scanner. github.com/cisco-ai-defense, 2026.

[10] Snyk. Agent Scan. github.com/snyk/agent-scan, 2026.

[11] A. Greensh. repo-forensics: Offline Security Scanner for AI-Agent Repos. github.com/alexgreensh/repo-forensics, 2026.

[12] K. Duraphe. trustcheck: Verify PyPI Package Attestations. github.com/kapilduraphe/trustcheck, 2026.

[13] M. J. F. Rehmani et al. DSSE: Dead Simple Signing Envelope. github.com/secure-systems-lab/dsse, 2020.

[14] Canari Contributors. MemTether: A Cross-Client AI Memory Hub. Internal reference, 2026.

[15] K. Tounoussi et al. MCP-Eco: A Linked Dataset of MCP Servers, Consumers, and Development Activity. arXiv:2611.06897, 2026. Data: huggingface.co/datasets/mcp-research-alliance/mcp-ecosystem.

[16] G. Chen et al. ProbeID: Verifying MCP Tool Identity for LLM Agents under Registry Drift. arXiv:2610.33633, 2026.